

Phase 4: Add Knowledge & Retrieval

This document provides the complete implementation for **Phase 4: Add Knowledge & Retrieval** for the **Banking — AI Banking Support & Advisory Agent (Non-Transactional)** project. In earlier phases: • Phase 1 defined the banking workflow, constraints, and user personas. • Phase 2 created a rule-based banking assistant. • Phase 3 upgraded the system using LLM reasoning and safety-aware prompts. Phase 4 improves the system further using Retrieval-Augmented Generation (RAG), semantic search, embeddings, and vector databases so the agent can answer banking questions using trusted internal banking knowledge.

Banking Safety Constraints

- The agent must never execute money movement.
- The agent must never approve loans or financial actions.
- The agent must avoid hallucinating banking policies.
- The agent must only answer using retrieved trusted documents.
- The agent must escalate ambiguous or missing-information cases.
- The system must avoid storing PII in embeddings or logs.

Why Retrieval-Augmented Generation (RAG) is Needed

The Phase 3 LLM-based system improved reasoning ability but still had a major weakness: the model could hallucinate banking information that did not exist in official policy documents. Examples: • Incorrect loan eligibility explanations • Invented interest-rate rules • Outdated banking policy information • Incorrect fraud-reporting procedures To solve this problem, Phase 4 introduces a retrieval system that allows the AI agent to search internal banking knowledge before generating responses.

1. RAG System Architecture

The Retrieval-Augmented Generation pipeline works as follows: User Query ↓ Embedding Generation ↓ Semantic Search in Vector Database ↓ Retrieve Relevant Banking Documents ↓ Pass Retrieved Context to LLM ↓ Generate Grounded Banking Response

2. Preparing Banking Documents for Embedding

Trusted banking documents must first be prepared before embeddings can be generated. Example knowledge sources: • Banking FAQ documents • Fraud prevention policies • Loan eligibility guides • Credit card policy manuals • Customer support SOPs • Compliance and escalation procedures

Large banking documents are split into smaller chunks so semantic search can retrieve the most relevant information accurately.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

documents = open("banking_policy.txt").read()

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_text(documents)
print(chunks[:2])
```

3. Generating Embeddings

Embeddings convert text into numerical vector representations. These vectors allow semantic similarity search instead of simple keyword matching.

```
from openai import OpenAI
client = OpenAI()

response = client.embeddings.create(
    model="text-embedding-3-small",
    input="How to report suspicious banking activity"
)

embedding = response.data[0].embedding

print(len(embedding))
```

4. Storing Embeddings in a Vector Database

The embeddings are stored in a vector database such as: • ChromaDB • FAISS • Pinecone • Weaviate These systems allow fast semantic retrieval of banking knowledge.

```
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings

embedding_model = OpenAIEmbeddings()

db = FAISS.from_texts(
    chunks,
    embedding_model
)

db.save_local("banking_faiss_index")
```

5. Implementing Semantic Search

Semantic search retrieves documents based on meaning instead of exact keywords. This greatly improves banking support quality.

```
query = "How do I report a fraudulent transaction?"

results = db.similarity_search(
    query,
    k=3
)

for doc in results:
    print(doc.page_content)
```

6. Connecting Retrieval Results to the Banking Agent

Retrieved banking documents are injected into the LLM prompt so the response is grounded in official banking knowledge.

```
def banking_rag_agent(user_query):

    docs = db.similarity_search(user_query, k=3)

    context = "\n".join(
        [doc.page_content for doc in docs]
    )
```

```

prompt = f'''
You are a Banking AI Support Assistant.

Use ONLY the provided banking documents.

Context:
{context}

Question:
{user_query}
'''

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "user", "content": prompt}
    ]
)

return response.choices[0].message.content

```

7. Comparing Responses With vs Without Retrieval

The same banking question was tested with and without retrieval. Question: "What is the official process for reporting debit card fraud?"

System	Response Quality	Problem
LLM Only	Generic fraud advice	May hallucinate policy details
RAG Banking Agent	Uses official fraud-reporting procedure	Grounded in internal banking documents

8. Handling Missing Information

A critical banking requirement is safely handling cases where relevant information cannot be found. The agent must NEVER hallucinate missing banking policies.

```

def safe_retrieval(query):

    docs = db.similarity_search(query, k=2)

    if not docs:
        return '''
        I could not find verified banking information
        for this request.

        Please contact a human banking representative.
        '''

    return docs

```

9. Retrieval Quality Testing

Retrieval systems must be evaluated systematically. Key evaluation metrics: • Retrieval relevance • Precision • Recall • Hallucination reduction • Response grounding quality • Latency

```

TEST_QUERIES = [
    "How do I report fraud?",
    "How can I reset my debit card PIN?",
    "What documents are required for a home loan?"
]

for query in TEST_QUERIES:

```

```
docs = db.similarity_search(query, k=3)

print("=" * 40)
print("QUERY:", query)

for d in docs:
    print(d.page_content[:200])
```

10. PII Protection in Retrieval Systems

Embedding pipelines must avoid storing sensitive customer data. Unsafe: • Account numbers • Phone numbers • PAN/Aadhaar details • Card numbers PII must be masked before embedding generation.

```
import re

def mask_sensitive_data(text):

    text = re.sub(r'\b\d{12}\b', '[MASKED_ACCOUNT]', text)
    text = re.sub(r'\b\d{16}\b', '[MASKED_CARD]', text)

    return text
```

11. Final Production RAG Banking Architecture

FINAL PHASE 4 ARCHITECTURE User Query ↓ PII Masking ↓ Embedding Generation ↓ Semantic Search ↓ Retrieve Banking Policies ↓ Inject Context into LLM ↓ Safety Validation ↓ Final Banking Response ↓ Escalation if Needed

Phase 4 significantly improved the Banking AI Support Agent by introducing retrieval-based reasoning and trusted banking knowledge access. The RAG architecture reduced hallucinations, improved policy accuracy, and ensured safer banking guidance while maintaining strict non-transactional banking constraints. The agent can now: • Search banking policies semantically • Answer using trusted documents • Reduce hallucinated responses • Escalate when information is missing • Maintain banking safety compliance